FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 3

Blade, Bootstrap et authentification

Durée : 5 jours Routes Contrôleurs Blade Bootstrap Sessions Sécurité

Objectifs de la semaine

- comprendre le trajet **route** → **contrôleur** → **vue** ;
- construire une mise en page commune avec **Blade** et **Bootstrap** ;
- écrire une **authentification** à la main et comprendre les sessions ;
- connaître trois protections de sécurité de base : CSRF, régénération de session, limitation des tentatives.

1. Routes et contrôleurs

```
// routes/web.php
Route::get('/login', [LoginController::class, 'create'])->name('login');
Route::post('/login', [LoginController::class, 'store']);
```

- `get` affiche une page, `post` reçoit un formulaire.
- `->name('login')` donne un nom à la route : dans le code on écrit `route('login')` plutôt que l'URL en dur.

```
php artisan make:controller Auth/LoginController
php artisan route:list
```

2. Blade : les gabarits

Un **layout** contient ce qui est commun à toutes les pages (menu, CSS, JS). Chaque page ne remplit que sa partie.

```
{{-- resources/views/layouts/app.blade.php --}}
<!doctype html>
<html lang="fr">
<head>
    <title>@yield('title', 'Tableau de bord')</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
    <nav class="navbar navbar-dark bg-dark">...</nav>
    <main class="container">@yield('content')</main>
</body>
</html>
```

```
{{-- resources/views/dashboard.blade.php --}}
@extends('layouts.app')

@section('content')
    <h1>Bonjour {{ auth()->user()->name }}</h1>
@endsection
```

> **Sécurité** — `{{ $variable }}` **échappe** le HTML : si quelqu'un saisit `<script>` dans un titre d'incident, il s'affichera comme du texte, sans s'exécuter. N'utilise `{!! !!}` que si tu sais exactement pourquoi.

Directives à connaître : `@if`, `@foreach`, `@forelse … @empty`, `@csrf`, `@method('PUT')`, `@error`, `@can`.

3. Bootstrap en 10 classes

Besoin	Classes
Grille	`container`, `row`, `col-md-6`, `g-3`
Carte	`card`, `card-header`, `card-body`
Tableau	`table`, `table-hover`, `table-responsive`
Boutons	`btn btn-primary`, `btn-outline-danger`, `btn-sm`
Formulaire	`form-label`, `form-control`, `form-select`, `is-invalid`, `invalid-feedback`
Badges, alertes	`badge text-bg-danger`, `alert alert-success`
Espacements	`mb-3` (marge bas), `p-4` (padding), `d-flex`, `gap-2`

4. Les composants Blade

Un composant est un petit morceau de vue réutilisable :

```
{{-- resources/views/components/badge.blade.php --}}
@props(['valeur'])
<span class="badge text-bg-{{ $valeur->color() }}">{{ $valeur->label() }}</span>
```

Utilisation : `<x-badge :valeur="$incident->severite" />`. Le `:` signifie « c'est une expression PHP ».

5. L'authentification

```
public function store(Request $request): RedirectResponse
{
    $identifiants = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (! Auth::attempt($identifiants, $request->boolean('remember'))) {
        return back()->withErrors(['email' => 'Identifiants incorrects.'])->onlyInput('email');
    }

    $request->session()->regenerate();

    return redirect()->intended(route('dashboard'));
}
```

- `Auth::attempt` cherche l'utilisateur et compare le mot de passe avec le **hash** stocké (jamais le mot de passe en clair).
- `session()->regenerate()` change l'identifiant de session à la connexion : protection contre la **fixation de session**.
- `redirect()->intended()` renvoie l'utilisateur vers la page qu'il voulait voir avant d'être redirigé vers la connexion.

Protéger des routes :

```
Route::middleware('auth')->group(function () {
    Route::get('/', DashboardController::class)->name('dashboard');
});
```

Les trois protections à connaître

1. **CSRF** : `@csrf` dans chaque formulaire `POST`. Sans lui, Laravel répond « 419 Page Expired ».
2. **Session** : régénérée à la connexion, invalidée à la déconnexion.
3. **Brute-force** : `->middleware('throttle:5,1')` sur la route de connexion = 5 essais par minute.

Exercice MySQL

Après t'être connecté, regarde la table `sessions` :

```
SELECT id, user_id, ip_address, FROM_UNIXTIME(last_activity) FROM sessions;
```

Déconnecte-toi et relance la requête. Que s'est-il passé ? Regarde aussi la colonne `password` de `users` : pourquoi ne peut-on pas retrouver le mot de passe ?

Travail demandé

1. Créer `layouts/app.blade.php` : barre de navigation (Tableau de bord, Incidents, Inventaire ▾, Rapport hebdo, Utilisateurs), nom de l'utilisateur, bouton Déconnexion, zone d'alertes flash.
2. Créer `LoginController` avec `create`, `store` et `destroy`. **Sans Breeze ni starter kit.**
3. Créer `auth/login.blade.php` (carte centrée sur fond sombre).
4. Organiser `routes/web.php` : groupe `guest` pour la connexion, groupe `auth` pour tout le reste.
5. Créer un tableau de bord provisoire qui affiche « Bonjour {nom} ».
6. Créer le composant `<x-badge>`.
7. Afficher les messages flash : `->with('success', '...')` côté contrôleur, `session('success')` dans le layout.

Livrable

Rien n'est accessible sans être connecté ; après connexion on arrive sur le tableau de bord avec le menu.

Auto-évaluation

- [ ] `/` sans être connecté redirige vers `/login`
- [ ] Mauvais mot de passe → message en français, l'e-mail reste pré-rempli
- [ ] La déconnexion se fait en `POST` avec `@csrf` (pas un simple lien)
- [ ] Après 5 essais ratés, la connexion est bloquée une minute
- [ ] Le menu indique la page active et se replie sur mobile
- [ ] Je sais expliquer à quoi sert le jeton CSRF

Ressources

- Laravel — Routing : laravel.com/docs/routing
- Laravel — Blade : laravel.com/docs/blade
- Laravel — Authentification manuelle : laravel.com/docs/authentication#authenticating-users
- Bootstrap 5.3 : getbootstrap.com/docs/5.3
- OWASP — CSRF : owasp.org/www-community/attacks/csrf

Pour aller plus loin

Ajouter « Se souvenir de moi » et expliquer le rôle du cookie `remember_token`.